

IDP-Lite Build Task List

Goal

Build a small Internal Developer Platform that lets a developer take a simple app from source code to a running Kubernetes service using CI, containers, GitOps, basic observability, and documented golden-path instructions.

Phase 1: Basic App

1. Create the demo app

- ☐ Create a new GitHub repo: `idp-lite-demo-app`
- ☐ Create a simple Node.js API
- ☐ Add `/health` endpoint
- ☐ Add `/ready` endpoint
- ☐ Add `/api/hello` endpoint
- ☐ Add basic unit test
- ☐ Add README with local run instructions

2. Run the app locally

- ☐ Install dependencies
 - ☐ Start the app locally
 - ☐ Test `/health`
 - ☐ Test `/ready`
 - ☐ Test `/api/hello`
-

Phase 2: Containerisation

3. Add Docker support

- ☐ Create `Dockerfile`
- ☐ Create `.dockerignore`
- ☐ Build Docker image locally
- ☐ Run Docker container locally
- ☐ Test `/health` through Docker

4. Add simple local commands

- ☐ Create `Makefile`
- ☐ Add `make install`
- ☐ Add `make test`
- ☐ Add `make build`
- ☐ Add `make run`
- ☐ Add `make docker-build`

- ☐ Add `make docker-run`
-

Phase 3: Kubernetes

5. Set up local Kubernetes

- ☐ Install or confirm Minikube is installed
- ☐ Start Minikube
- ☐ Confirm cluster is running
- ☐ Confirm `kubectl` can connect to the cluster

6. Add Kubernetes manifests

- ☐ Create `k8s/` folder
 - ☐ Add `deployment.yaml`
 - ☐ Add `service.yaml`
 - ☐ Add liveness probe
 - ☐ Add readiness probe
 - ☐ Deploy app to Minikube
 - ☐ Confirm pod is running
 - ☐ Port-forward service
 - ☐ Test `/health` from Kubernetes
-

Phase 4: CI Pipeline

7. Add GitHub Actions

- ☐ Create `.github/workflows/ci.yml`
- ☐ Add checkout step
- ☐ Add dependency install step
- ☐ Add lint step
- ☐ Add test step
- ☐ Add Docker build step
- ☐ Push code and confirm pipeline passes

8. Add container registry

- ☐ Use GitHub Container Registry
 - ☐ Add image build step to CI
 - ☐ Add image push step to CI
 - ☐ Tag image with commit SHA
 - ☐ Confirm image appears in registry
-

Phase 5: GitOps

9. Create platform config repo

- ☐ Create new GitHub repo: `idp-lite-platform-config`
- ☐ Add `apps/demo-app/` folder
- ☐ Move Kubernetes manifests into platform config repo
- ☐ Update image reference to use GitHub Container Registry image
- ☐ Commit and push platform config repo

10. Install ArgoCD

- ☐ Install ArgoCD in Minikube
- ☐ Access ArgoCD UI
- ☐ Log in to ArgoCD
- ☐ Create ArgoCD application manifest
- ☐ Point ArgoCD at `idp-lite-platform-config`
- ☐ Sync the demo app
- ☐ Confirm app is `Healthy`
- ☐ Confirm app is `Synced`

11. Prove the GitOps loop

- ☐ Make a small app change
 - ☐ Push change to app repo
 - ☐ Confirm CI builds and pushes new image
 - ☐ Update image tag in platform config repo
 - ☐ Confirm ArgoCD deploys the new version
 - ☐ Test the deployed app
-

Phase 6: Developer Experience

12. Add golden-path documentation

- ☐ Create `docs/` folder
- ☐ Add `getting-started.md`
- ☐ Add `golden-path.md`
- ☐ Add `troubleshooting.md`
- ☐ Add `architecture.md`

13. Improve developer commands

- ☐ Add `make deploy-local`
- ☐ Add `make logs`
- ☐ Add `make status`
- ☐ Add `make port-forward`
- ☐ Add `make clean`
- ☐ Document all commands in README

14. Add architecture diagram

- ☐ Draw simple flow diagram
 - ☐ Show app repo
 - ☐ Show CI pipeline
 - ☐ Show container registry
 - ☐ Show platform config repo
 - ☐ Show ArgoCD
 - ☐ Show Kubernetes cluster
 - ☐ Add diagram to `architecture.md`
-

Phase 7: Observability

15. Add basic app-level observability

- ☐ Add structured logging
- ☐ Confirm logs are visible with `kubectl logs`
- ☐ Add `/metrics` endpoint if using Prometheus-compatible metrics
- ☐ Document how to view app logs

16. Add cluster-level observability

- ☐ Install Prometheus
 - ☐ Install Grafana
 - ☐ Confirm app metrics are being scraped
 - ☐ Create or import a simple dashboard
 - ☐ Document how to access Grafana
-

Phase 8: Quality and Security Gates

17. Add quality checks

- ☐ Add linting
- ☐ Add unit tests
- ☐ Add API smoke test
- ☐ Add CI step for API smoke test
- ☐ Fail pipeline if tests fail

18. Add security checks

- ☐ Add Trivy container scan
 - ☐ Add dependency scan
 - ☐ Add Kubernetes manifest validation with `kubeconform`
 - ☐ Add these checks to CI
 - ☐ Document what each check does
-

Phase 9: Self-Service Feature

19. Create a service template

- ☐ Create a template folder for new services
- ☐ Include starter app structure
- ☐ Include Dockerfile
- ☐ Include Kubernetes manifests
- ☐ Include GitHub Actions workflow
- ☐ Include README template

20. Add scaffold script

- ☐ Create `create-service.sh`
 - ☐ Script accepts service name
 - ☐ Script creates folder structure
 - ☐ Script replaces placeholder app name
 - ☐ Test creating a new service from the template
-

Phase 10: Final Polish

21. Create final demo flow

- ☐ Start from a clean repo
- ☐ Make app change
- ☐ Push change
- ☐ CI runs
- ☐ Image builds
- ☐ Image is pushed
- ☐ Platform config is updated
- ☐ ArgoCD syncs
- ☐ App is deployed
- ☐ Logs and metrics are visible

22. Write portfolio summary

- ☐ Write short project description
 - ☐ Explain the problem the IDP-lite solves
 - ☐ Explain the golden path
 - ☐ Explain the tools used
 - ☐ Explain quality/security gates
 - ☐ Explain what you would improve next
-

Definition of Done

The IDP-lite is complete when:

- ☐ A simple app can run locally

- ☐ The app can run in Docker
- ☐ The app can run in Kubernetes
- ☐ CI runs tests and builds the image
- ☐ Image is pushed to a container registry
- ☐ ArgoCD deploys the app through GitOps
- ☐ Basic logs and metrics are available
- ☐ Quality and security checks exist in CI
- ☐ Golden-path documentation exists
- ☐ A simple service template or scaffold script exists
- ☐ You can demo the full path end-to-end